

LLM as Judges for Coding Tasks: Robustness and Evaluation

Team: Neuronauts

Mentor:
Dr. Neeraj Varshney

Megha Suresh
Arizona State University

Nandana Vikraman
Arizona State University

Avantika Tiwari
Arizona State University

Jahnavi Krishna Kovvuri
Arizona State University

Amulya Nekkanti
Arizona State University

1.Problem Statement

Large Language Models (LLMs) are increasingly employed as automated judges in software engineering workflows, where they are used to assess code quality, evaluate translations, and compare candidate solutions. However, their reliability in such evaluative roles remains insufficiently understood. Prior studies indicate that LLM judges may exhibit systematic vulnerabilities including positional bias, bandwagon bias, and instability under reasoning-based prompting which can lead to inconsistent or incorrect decisions.

A critical gap in current evaluation practice is the absence of comprehensive robustness analysis. Existing benchmarks focus primarily on correctness but rarely examine whether models make semantically grounded judgments or whether their decisions are unduly influenced by prompt structure or contextual framing. Understanding these limitations is essential before LLM judges can be trusted in high-stakes or automated evaluation pipelines.

This work systematically evaluates four modern open-source LLMs; Qwen2.5, Qwen3-8B, LLaMA-8B, and Gemma-3B on two representative judgment tasks: code summarisation and code translation. Models are assessed across multiple inference strategies (baseline, Chain-of-Thought, self-verification, and Best-of-N) and under controlled perturbations designed to measure susceptibility to position bias and bandwagon bias. Through this analysis, the study provides an empirical characterization of the strengths and limitations of current LLM-as-a-Judge systems.

Finally, the project establishes the foundation for fine-tuning a specialized, bias-resistant LLM judge that can be shared with the community to enable more consistent, transparent, and reproducible automated code evaluation.

2.Approach to Address the Problem

Our approach is built around constructing controlled variants of coding tasks, turning them into pairwise A/B judgment problems, and systematically benchmarking multiple LLMs as judges under different evaluation settings.

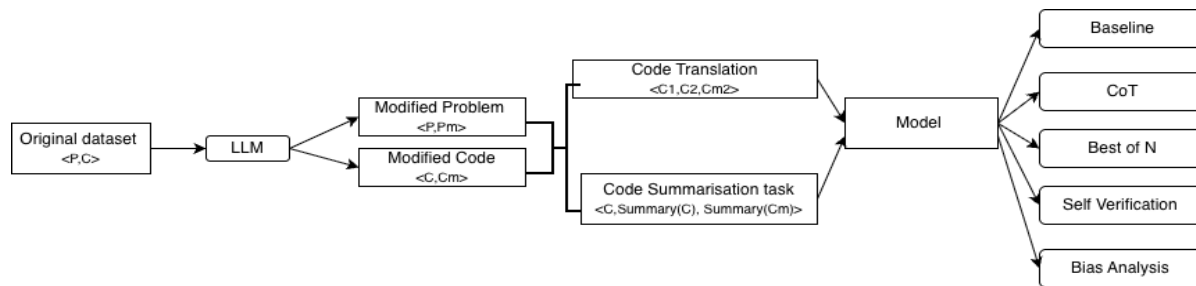


Figure1: The basic pipeline of the project.

2.1 Construct Variants of Each Task (P,C, Pm, Cm)

For every seed task code summarization or code translation we start from a problem with a known correct solution:

- **P**: original natural-language problem description.
- **C**: verified correct solution code for P.
- From each seed pair (P, C), we generate structured variants using two different models:
- **Pm – Modified problem inputs (Gemma 3B)**. Pm is generated using **Gemma 3B** with *only* the following controlled transformations:

- **Minor Constraint Modification** – adjusting limits or ranges (e.g., changing bounds).
- **Edge-Case Extension** – adding explicit boundary or corner cases to the problem description.
- **Intent Preservation with Parameter Change** – renaming or re-scaling parameters while preserving the overall goal.
- **Objective Swap** – swapping the task objective (e.g., minimize ↔ maximize, count ↔ check).
- **Cm – Incorrect code variants (Qwen/Qwen2.5).** Cm is generated using **Qwen/Qwen2.5** by introducing controlled semantic bugs into solutions for P. We restrict ourselves to:
 - **Conditional Logic Change** – modifying branch conditions or if/else structure.
 - **Logical Inversion** – flipping comparison or boolean logic (e.g., < to <=, == to !=).
 - **Off-by-One Error** – shifting indices or loop bounds by ±1.
 - **Partial Implementation** – omitting parts of the required logic.
 - **Removed Edge-Case Handling** – deleting checks for rare or boundary inputs.

These controlled variants let us test whether a judge focuses on **functional correctness** rather than being fooled by small but critical semantic changes. They also realize the first stage of our pipeline:

2.2 Build Pairwise A/B Judgment Tasks

Next, we convert the above variants into **pairwise A/B judgment tasks**:

- For each instance, we form a pair where:
 - One candidate is correct
 - The other candidate is incorrect or adversarially perturbed
- The judge model receives:
 - the task input (problem description and/or code snippet),
 - two outputs, **A** and **B**, and must choose which candidate is correct.

Code Summarisation: For summarisation tasks, the model receives a code snippet and two candidate summaries. Candidate A or B contains the ground-truth summary derived from the correct solution *C*, while the other candidate is produced by summarising the incorrect variant *Cm*. This ensures that the incorrect option remains syntactically plausible but semantically misleading.

Code Translation: For translation tasks, the model is given a source-language program and two candidate translations. The correct candidate is a faithful translation of the original code, whereas the incorrect candidate is derived from translating *Cm* or generated through controlled perturbations that break semantic equivalence. The judge must identify which translation preserves the original program logic.

2.3 Benchmark LLM Judges Under Diverse Settings

We benchmark multiple LLMs as judges on these A/B tasks under several judgment styles.

Initial model exploration:

We first ran a pilot study with several LLM families including **GPT models**, **DeepSeek**, **Mistral**, and similar open models on a subset of tasks. This helped us understand their behaviour, latency, cost and reproducibility. Based on these experiments and resource constraints, we moved to an open, fully reproducible stack for final evaluation.

Final judge model set (proposed judge configuration):

All main experiments use the following open-source judge models:

- **Qwen/Qwen2.5-Coder-7B-Instruct** – primary code-specialized judge.
- **Qwen/Qwen3-8B** – larger Qwen-family model used across all analyses.
- **Llama-3.1-8B-Instruct** – general instruction-tuned baseline.
- **Gemma 3-4B-it** – compact instruction-tuned judge.

Judgment styles:

Each model is evaluated under multiple prompting strategies:

- **Baseline:** simple A/B choice.
- **Self-Verification:** the model is asked to think/check before deciding.
- **Best-of-N:** multiple judgments are sampled; a majority vote is taken.
- **CoT / Zero-Shot Reasoning:** the model reasons step-by-step without task-specific examples.

Common inference protocol:

Across all models and settings we use a shared protocol:

- Fixed, task-specific prompts per judgment style.
- Input is always the task description plus A/B candidates.
- The final decision recorded for evaluation is **only the label “A” or “B”**.
- Shared decoding settings ensure deterministic and comparable results.

The combination of this dataset pipeline, the final judge model set, and the judgment styles described above constitutes our proposed LLM-as-a-Judge framework.

2.4 Validate Perturbations and Analyze Bias Susceptibility

Finally, we analyse how robust these judge models are to the perturbations we introduce and what kinds of examples they fail on.

Position bias analysis: For each A/B example, we also create a version where the candidates are swapped. We then measure accuracy separately for cases where the correct answer appears in position A and where it appears in position B, across all judge models. The flip rate represents the model’s position bias, showing how much its decision is influenced by the order of the options rather than their content.

Bandwagon Bias Analysis: To evaluate susceptibility to conformity bias, we augment prompts with a synthetic “crowd preference” signal, indicating that previous annotators mostly preferred either option A or option B

3.Results Obtained from the Chosen Approach

This section summarizes the performance of four LLMs Qwen2.5, LLaMA, Qwen3-8B, and Gemma across two tasks: code summarization and code translation. Each model was evaluated under multiple inference settings: baseline, self-verification, Best-of-N sampling, and Chain-of-Thought. Bias analysis with bandwagon bias and position bias.

3.1 Code Summarization

- Among all models, **Qwen3-8B** achieved the highest baseline accuracy, followed by Qwen2.5 and Gemma. LLaMA performed the weakest. This indicates that larger instruction-tuned models generalize better for summarization-based judging tasks.
- Self-verification degraded performance for all models. This shows that self-verification does not consistently correct errors. Instead, models often overwrite correct initial judgments with inconsistent reasoning.
- Best-of-N helped LLaMA (0.6226 to 0.6509) and Gemma (0.6447 to 0.6541) but showed marginal gains or declines for Qwen models.
- **Gemma** saw the largest improvement in **zero-shot CoT** (0.6447 to 0.8467). This suggests CoT benefits models that are trained with reasoning-style datasets (e.g., Gemma), but may hurt models that generate overly verbose or incorrect chains.
- Introducing a bandwagon signal reduced accuracy for all models. The substantial drop for Gemma (−16%) and LLaMA (−6.6%) highlights high susceptibility to conformity bias.
- All models demonstrated measurable position bias during code summarisation, with Qwen2.5, LLaMA, and Gemma showing a strong preference for choosing the answer in position A (large accuracy gaps between A and B), while Qwen3-8B was the least affected but still slightly favored A indicating that model judgments are influenced not only by candidate quality but also by where the answer is placed.

Code Summarization				
Analysis	Qwen 2.5	LLama	Qwen 3 8b	Gemma
Baseline	0.6855	0.6226	0.7987	0.6447
Self-verification	0.6509	0.5503	0.739	0.6824
BestofN	0.6415	0.6509	0.783	0.6541
CoT- ZeroShot	0.6981	0.6384	0.7767	0.8467

Table 1: Analysis Summary for Code Summarisation task

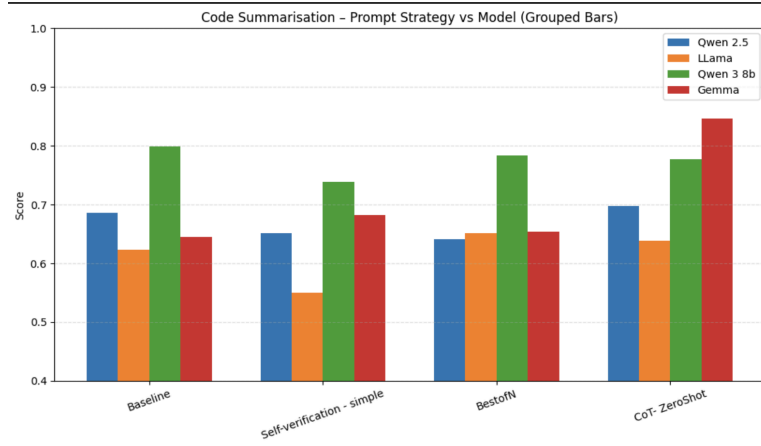


Figure 2: Code Summarisation Analysis

Code Summarization				Code Summarization		
Model Name	Baseline	Accuracy at A	Accuracy at B	Model Name	Baseline	Bandwagon
Qwen 3	0.7987	0.805	0.7924	Qwen 3	0.7987	0.7642
Qwen 2.5	0.6855	0.7987	0.5597	Qwen 2.5	0.6855	0.67
LLama	0.6226	0.8931	0.3522	LLama	0.6226	0.5566
Gemma	0.6447	0.7421	0.5472	Gemma	0.6447	0.4843

Table 2: Result of Bias Analysis

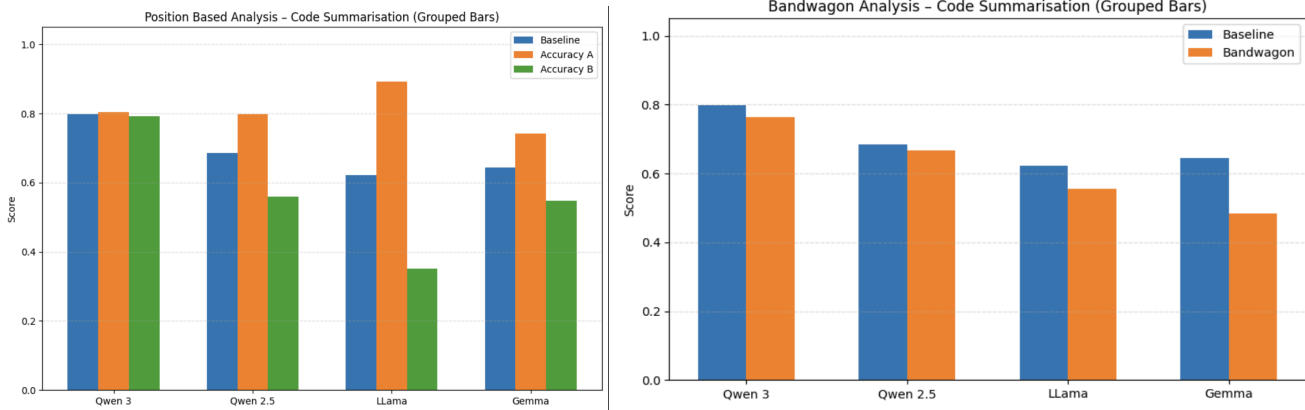


Figure 3: Bias Analysis for Code Summarization

3.2. Code Translation

- **Baseline Performance:** **Translation accuracy was significantly higher than summarization:** Qwen2.5: 0.8767, Qwen3-8B: 0.8733, Gemma: 0.7600. The stronger performance reflects that translation tasks are more deterministic and easier to judge models.
- **Self-verification improved results for all models:** Qwen3-8B: 0.8733 to 0.9333, Qwen 2.5: 0.8767 to 0.8833, Gemma: 0.7600 to 0.8300. This is the opposite trend compared to summarization. Because translation is structured and less ambiguous, verification allows the model to double-check literal correctness, leading to improvements.
- **Best-of-N improved Qwen 2.5** to 0.923, the highest across all settings. Qwen3-8B decreased (0.8733 to 0.8467), showing strong initial confidence already.
- **CoT generally helped translation:** Qwen3 zero-shot CoT: 0.8733 to 0.8967, Gemma zero-shot CoT: 0.7600 to 0.8467

- All models experienced **substantial declines** in bandwagon analysis: Qwen3-8B: 0.8733 to 0.7367, Qwen2.5: 0.8767 to 0.7967, LLaMA: 0.5867 to 0.4900. This reaffirms that judge LLMs are highly vulnerable to social influence cues, even in deterministic translation tasks.
- In code translation, models again showed **strong position bias**, with Qwen3-8B and Qwen2.5 preferring answers in position B, LLaMA displaying extreme preference for position A, and Gemma remaining almost position-neutral, indicating that even in deterministic tasks, model judgments are still influenced by candidate ordering.

Code Translation				
Analysis	Qwen 2.5	LLama	Qwen 3 8b	Gemma
Baseline	0.8767	0.5867	0.8733	0.76
Self - Verification	0.8833	0.76	0.933333	0.83
BestofN	0.923	0.6633	0.8467	0.7567
CoT- Zero Shot	0.89	0.6	0.8967	0.8467

Table 4: Analysis Summary of Code Translation

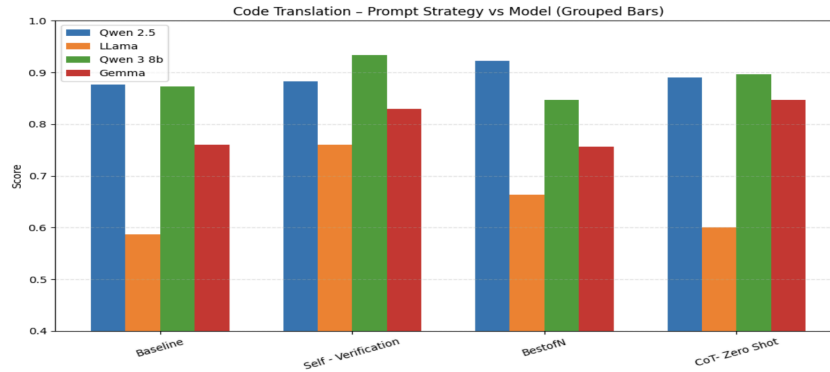


Fig. 4. Analysis for Code Translation

Code Translation				Code Translation		
Model Name	Baseline	Accuracy at A	Accuracy at B	Model Name	Baseline	Bandwagon
Qwen 3	0.8733	0.75333	0.99333	Qwen 3	0.8733	0.7367
Qwen 2.5	0.8767	0.8533	0.9	Qwen 2.5	0.8767	0.7967
LLama	0.5867	1	0.24	LLama	0.5867	0.49
Gemma	0.76	0.9067	0.9067	Gemma	0.76	0.5267

Table 5: Bias Analysis Results for Code Translation

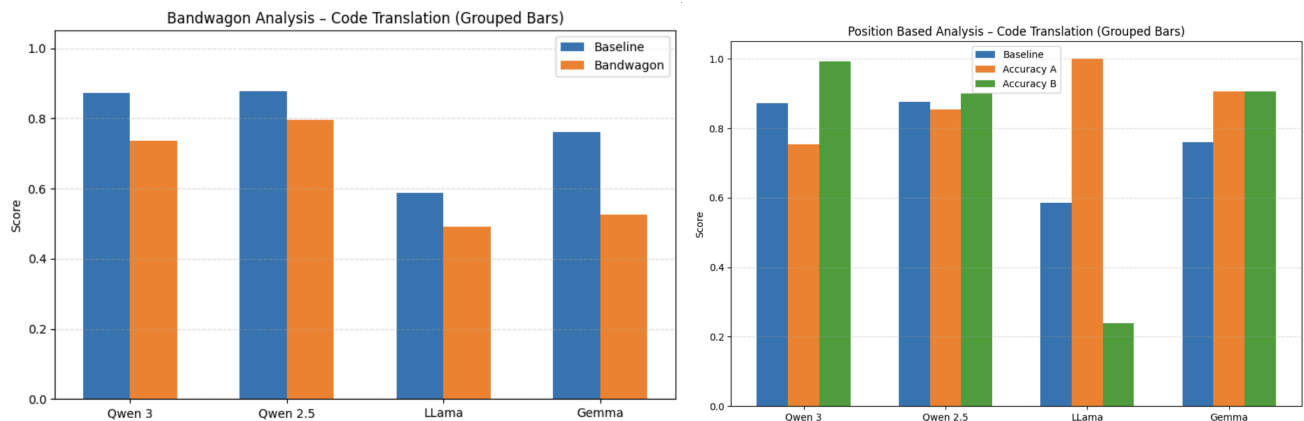


Fig. 5. Bias Analysis for Code Translation

4. Analysis of Results

Across both code summarisation and code translation tasks, **Qwen3-8B consistently demonstrates the strongest overall judgment capability**, outperforming all other models under baseline and most prompting strategies. Its higher accuracy

suggests that larger, instruction-aligned models possess more stable internal representations for semantic code understanding, making them more reliable judges in structured decision-making settings. In contrast, **LLaMA-8B repeatedly exhibits the weakest performance**, indicating limitations in its ability to generalize to fine-grained code evaluation tasks, despite being a capable instruction-following model for natural language.

A consistent trend observed across models is their **susceptibility to bandwagon bias**. All models exhibit noticeable accuracy drops when exposed to synthetic “crowd preference” cues, implying that LLM judges are sensitive to external framing even when the bias is irrelevant to the underlying decision. **Position bias** is also prominently visible. LLaMA shows the most extreme positional skew, strongly favoring candidates presented in position A, with accuracy dropping sharply when the correct option appears in B. The effects of **self-verification** exhibit a task-dependent pattern. In summary, self-verification generally **reduces accuracy**, likely due to models revising correct initial predictions when attempting to re-evaluate with additional reasoning steps. This suggests that, for more open-ended or interpretive tasks, additional reasoning may introduce noise rather than clarity. In contrast, in translation tasks, self-verification **improves accuracy**, implying that structured transformations such as translation benefit from explicit second-pass checking, where the model can more effectively verify semantic equivalence between the source and target.

The impact of **Best-of-N sampling** further reinforces the differences between strong and weak models. Best-of-N yields noticeable improvements for weaker models such as LLaMA and Gemma, which benefit from multiple independent reasoning paths that help smooth out inconsistent single-shot predictions. For more capable models like Qwen3-8B, the gains are smaller or occasionally negative, indicating that the model’s single-shot predictions are already close to its performance and additional samples may introduce unnecessary variance.

Overall, these results reveal that while advanced LLMs can serve as competent automated judges, their behavior is highly dependent on model architecture, task type, prompting strategy, and input framing. The consistent presence of bias effects and the variable performance across reasoning modes underscore the need for specialized, fine-tuned judge models designed to minimize positional and framing sensitivities while maximizing semantic reliability.

5. Individual Contributions of Team Members

- **Amulya Nekkanti:** Contributed to the Code Translation Benchmarking task by running evaluation experiments across multiple prompting strategies and assisting in analyzing model behavior. Worked on the Project Proposal and helped in building and refining the translation dataset through manual verification and cleaning.
- **Jahnvi Krishna Kovvuri:** Worked extensively on Code Translation Benchmarking, helping generate A/B evaluation pairs and analyze judge performance under different settings. Contributed to the Project Proposal and supported dataset creation, ensuring high-quality translation examples.
- **Avantika Tiwari:** Worked on Code Summarization Benchmarking by preparing summarization datasets and evaluating LLM judge performance on summarization tasks. Contributed to Project Phase 1 and participated in dataset creation using the unified formatting pipeline.
- **Nandana Vikraman:** Contributed to Code Summarization Benchmarking and assisted in Project Phase 1 by preparing summarization samples and testing various prompting strategies. Worked on dataset creation, focusing on ensuring accuracy and consistency across summarization tasks.
- **Megha Suresh:** Contributions to the project include developing the unified dataset creation pipeline for summarisation, translation, and code-pair evaluation tasks.. Work also involved building the benchmarking framework covering prompt construction, multiple inference variants (baseline, CoT, self-verification, Best-of-N, and bias-conditioned prompts), and evaluation scripts and running large-scale experiments across Qwen2.5, Qwen3-8B, LLaMA, and Gemma models.

6. Ongoing Work: Validation and Fine-Tuning

We are currently conducting detailed error analysis on the model’s predictions. Based on the insights from this analysis, we are fine-tuning the model parameters using the train dataset generated to improve performance and reduce systematic errors.

7. Acknowledgement

We express our sincere gratitude to Dr. Neeraj Varshney for his constant guidance, insightful feedback, and support throughout the project.

References

- H. Jiang, Y. Chen, Y. Cao, H.-Y. Lee, and R. T. Tan. “CodeJudgeBench: Benchmarking LLM-as-a-Judge for Coding Tasks.” arXiv preprint arXiv:2502.10533, 2025.
- Z. Wang, Y. Zhou, X. Zhou, D. Lu, and T. Chen. “CODE-EDITING: A Reasoning Benchmark for Functional Alignment in Code.” arXiv preprint arXiv:2502.11022, 2025.
- L. Bishay, B. Shrestha, M. Abdelnabi, V. Sharma, C. Wauthier, R. A. Popa, and T. Brunschweiler. “Phoenix: A Framework for Evaluating LLMs as Judges.” In Proc. International Conference on Learning Representations (ICLR), 24.
- X. Zou, K. Kim, T. Zhang, M. Weyssow, L. F. Gomes, G. Yang, and D. Lo. “An Automatic Metric for Bridging the Gap with Human Evaluation in SE Tasks.” ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 1, no. 1, p. 1, Feb. 2025, doi: 10.1145/XXXXXXX.XXXXXXX.
- T.-H. Tsai, Y. Zhuo, Z. Teddu, J. Sun, Z. Xing, X. Zhu, and D. Lo. “From Code to Contracts: LLMs as the New Software Judges.” arXiv preprint arXiv:2503.02246, 2025.

Appendix

A: Project Lifecycle

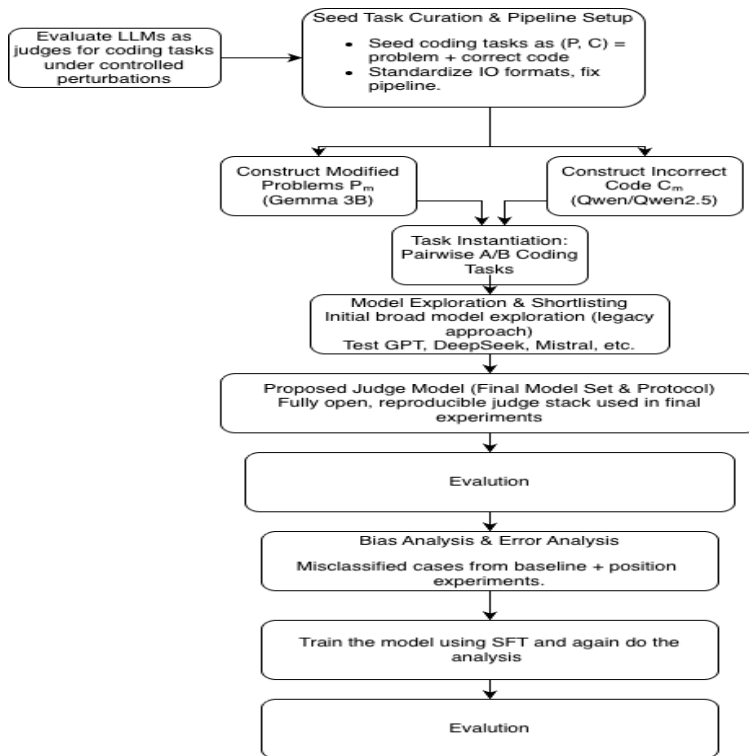


Figure 6 : Lifecycle of the project

Figure 6 illustrates the overall lifecycle followed in this project, from dataset construction to model evaluation. The pipeline begins with seeding initial coding tasks and generating modified problem and code variants (P_m and C_m) using LLMs. These variants are then used to construct pairwise A/B judgment tasks for summarisation and translation, enabling systematic benchmarking of multiple judge models. The final stage involves evaluating prompting strategies, analysing bias susceptibility, and performing error inspection. This lifecycle supports a reproducible framework for understanding the robustness and limitations of LLM-as-a-Judge systems. After identifying failure patterns and bias sensitivities in the baseline evaluation, the next stage involves fine-tuning the model using Supervised Fine-Tuning

(SFT) on the curated A/B judgment dataset. The objective is to help the model internalize decision boundaries for correct vs. incorrect summaries and translations, reduce susceptibility to position and bandwagon biases, and improve consistency across prompting styles. Once fine-tuned, the model is re-evaluated using the same inference protocols baseline, CoT, self-verification, and Best-of-N to measure improvements relative to the original benchmarks. This closed-loop analysis provides quantitative evidence of whether fine-tuning enhances judgment accuracy, robustness, and bias resistance.

B. Instruction Fine-Tuning (IFT) for Enhancing LLM-as-a-Judge

- To further improve the reliability, robustness, and consistency of LLM judges, we propose applying Instruction Fine-Tuning (IFT) on the datasets created in this project. While our current work evaluates off-the-shelf LLMs, IFT enables us to transform a strong coding model (e.g., Qwen 2.5 or Qwen 3 8B) into a domain-specialized judge trained specifically on summarization and translation assessment tasks.
- IFT allows the model to learn how to evaluate paired outputs (A and B), identify subtle semantic differences, justify reasoning, resist stylistic or bias-inducing cues, and deliver consistent judgments across varied adversarial conditions.

C. Prompt Template Used in the Evaluation

1) Code Summarization

```
You are given a code snippet and two candidate summaries (A and B).  
Choose which summary correctly describes the behavior of the code.  
Output the final answer as:  
Final Label: A or Final Label: B
```

```
Code:  
def count_divisible_by_7_not_5(nums):  
    result = 0  
    for n in nums:  
        if n % 7 == 0 and n % 5 != 0:  
            result += 1  
    return result
```

Candidate A:

This function counts how many integers in the list are divisible by seven while not being divisible by five.

Candidate B:

This function counts how many integers in the list are divisible by five while not being divisible by seven.

Final Label: A

2) Code Translation

```
You are given a source program and two candidate translations (A and B).  
Choose the translation that correctly preserves the logic and behavior of the  
original code.
```

```
Output the final answer as:
```

```
Final Label: A or Final Label: B
```

```
Source Code (S):
```

```
def is_palindrome(s):  
    s = s.lower()  
    return s == s[::-1]
```

```
if __name__ == "__main__":
```



```

s = "Level"
print("YES" if is_palindrome(s) else "NO")

```

Candidate A:

```

#include <iostream>
#include <string>
#include <cctype>

bool isPalindrome(const std::string& s) {
    std::string lower;
    lower.reserve(s.size());
    for (char c : s) {
        lower.push_back(static_cast<char>(
            std::tolower(static_cast<unsigned char>(c))
        ));
    }
    int i = 0;
    int j = static_cast<int>(lower.size()) - 1;
    while (i < j) {
        if (lower[i] != lower[j]) {
            return false;
        }
        ++i;
        --j;
    }
    return true;
}

int main() {
    std::string s = "Level";
    std::cout << (isPalindrome(s) ? "YES" : "NO") << std::endl;
    return 0;
}

```

Candidate B:

```

#include <iostream>
#include <string>

bool isPalindrome(const std::string& s) {
    int i = 0;
    int j = static_cast<int>(s.size()) - 1;
    while (i < j) {
        if (s[i] != s[j]) { // wrong: case-sensitive comparison
            return false;
        }
        ++i;
        --j;
    }
    return true;
}

int main() {

```

```
std::string s = "Level";  
std::cout << (isPalindrome(s) ? "YES" : "NO") << std::endl;  
return 0;  
}  
Final Label: A
```